

K8S-02: DynaKube Operator

Deployment

Series: K8S — Kubernetes Monitoring | **Notebook:** 2 of 13 | **Created:** January 2026
| **Last Updated:** 07/24/2026

Installing and Configuring the Dynatrace Operator

The DynaKube operator is the recommended way to deploy Dynatrace monitoring in Kubernetes. This notebook covers installation via Helm, configuration options, and deployment modes for different use cases.

Table of Contents

1. [Prerequisites Setup](#)
 2. [Helm Chart Installation](#)
 3. [DynaKube Custom Resource](#)
 4. [Deployment Modes Explained](#)
 5. [Configuration Options](#)
 6. [Verification and Validation](#)
 7. [Upgrading the Operator](#)
-

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS tenant with API tokens
Kubernetes Cluster	v1.24+ with admin access
Helm	v3.x installed
kubectl	Configured for your cluster
API Tokens	Operator token (Platform Token recommended) + Data ingest token

Operator token type: new automation should use **Platform Tokens** (`dt0s16`) with `Authorization: Bearer` for the Operator token — note that **the Operator itself accepts platform tokens only from Operator 1.10.0** (released July 15, 2026): *"No immediate action is required and existing access tokens continue to be accepted."* On clusters running an earlier Operator, keep the classic access (API) token — it remains the working path. The OneAgent installer download path still uses Classic API Tokens (`dt0c01` with `Authorization: Api-Token`); this is expected and supported. See the [IAM series](#) for parameterized policy patterns and the ONBRD-99 Recommended Defaults card for the canonical token / configuration / extension stack.

Version Support Policy: OneAgent and ActiveGate versions are supported for **9 months (Standard)** or **12 months (Enterprise)**. Third-party technologies are supported for 6 months beyond vendor EOL. See [Support Policy \(Dynatrace\)](#).

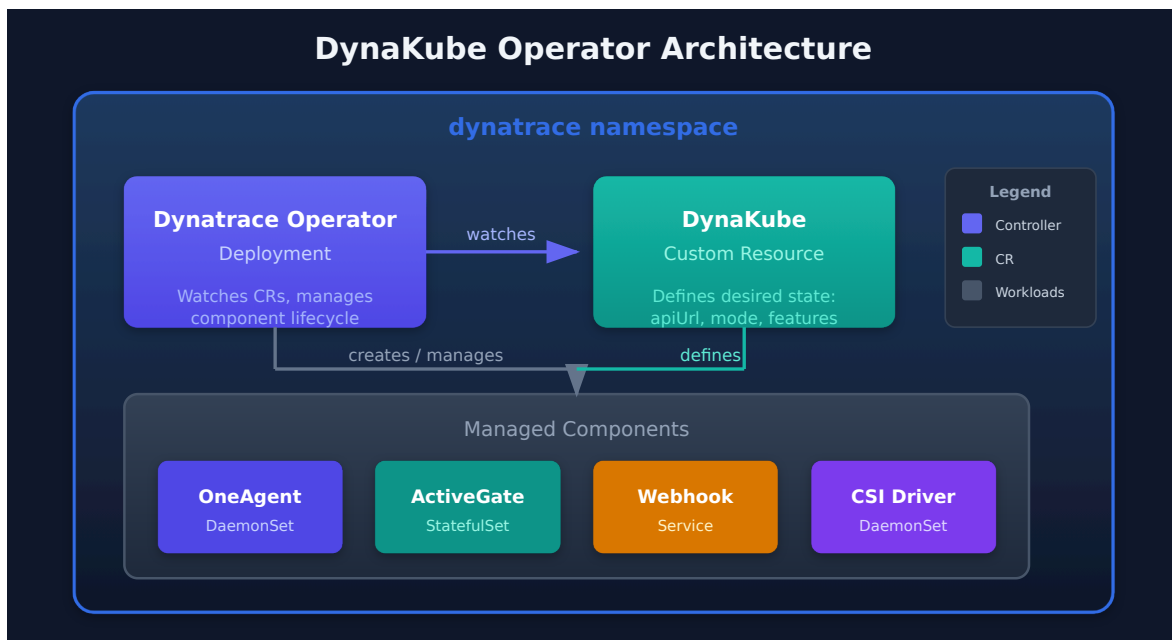
1. Operator Overview

The Dynatrace Operator manages the complete lifecycle of Dynatrace monitoring components.

What the Operator Manages

Component	Description	Managed By
OneAgent DaemonSet	Node-level monitoring	Operator
ActiveGate StatefulSet	Routing and K8s API access	Operator
Webhook	Code module injection	Operator
CSI Driver	Volume-based code modules	Operator

Operator Architecture



2. Prerequisites Setup

Required API Tokens

Create two tokens in Dynatrace with these scopes:

Operator Token:

Scope	Purpose
<code>activeGateTokenManagement.create</code>	ActiveGate tokens

Scope	Purpose
entities.read	Entity information
settings.read	Read settings
settings.write	Write settings
DataExport	Export data (optional)
InstallerDownload	Download OneAgent

Data Ingest Token:

Scope	Purpose
metrics.ingest	Ingest metrics
logs.ingest	Ingest logs
events.ingest	Ingest events (optional)

Create Namespace and Secret

```
# Create namespace
kubectl create namespace dynatrace

# Create secret with tokens
kubectl create secret generic dynakube \
  --namespace dynatrace \
  --from-literal=apiToken=<OPERATOR_TOKEN> \
  --from-literal=dataIngestToken=<DATA_INGEST_TOKEN>
```

3. Helm Chart Installation

Add the Dynatrace Helm Repository

```
# Add Dynatrace Helm repo
helm repo add dynatrace https://raw.githubusercontent.com/Dynatrace/dynatrace-operator/main/config/helm/repos/stable

# Update repo cache
helm repo update

# View available versions
helm search repo dynatrace/dynatrace-operator --versions
```

Install the Operator

```
# Install operator only (DynaKube CR applied separately)
helm install dynatrace-operator dynatrace/dynatrace-operator \
  --namespace dynatrace \
  --create-namespace \
  --wait
```

Install with Custom Values

```
# Create values file
cat > dt-values.yaml <<< 'EOF'
# Operator configuration
operator:
  image: ""
  customPullSecret: ""

# Platform-specific settings
platform: "kubernetes" # or "openshift"

# Webhook configuration
webhook:
  hostNetwork: false

# CSI driver
csidriver:
  enabled: true
EOF

# Install with values
helm install dynatrace-operator dynatrace/dynatrace-operator \
  --namespace dynatrace \
  --values dt-values.yaml \
  --wait
```

4. DynaKube Custom Resource

The DynaKube CR defines how monitoring is deployed.

Minimal DynaKube (cloudNativeFullStack)

```
apiVersion: dynatrace.com/v1beta5
kind: DynaKube
metadata:
  name: dynakube
  namespace: dynatrace
spec:
  # Your Dynatrace environment URL
  apiUrl: https://ENVIRONMENT_ID.live.dynatrace.com/api

  # OneAgent deployment mode
  oneAgent:
    cloudNativeFullStack:
      tolerations:
        - effect: NoSchedule
          key: node-role.kubernetes.io/master
          operator: Exists
        - effect: NoSchedule
          key: node-role.kubernetes.io/control-plane
          operator: Exists

  # ActiveGate for Kubernetes monitoring
  activeGate:
    capabilities:
      - kubernetes-monitoring
      - routing
      - dynatrace-api
```

API Version Note: `v1beta6` is the current DynaKube API version (introduced in Operator 1.8.0). `v1beta5` remains supported and all examples in this series use it. When starting new deployments, consider using `v1beta6`.

Apply the DynaKube

```
# Apply DynaKube CR
kubectl apply -f dynakube.yaml

# Watch deployment progress
kubectl -n dynatrace get dynakube -w
```

5. Deployment Modes Explained

cloudNativeFullStack (Recommended)

```
oneAgent:
  cloudNativeFullStack:
    # Injected via webhook using CSI driver volumes
```

Pros	Cons
No privileged containers for apps	Requires CSI driver
Best for multi-tenant clusters	Slightly more complex
Independent app/infra monitoring	

classicFullStack (Supported — Avoid for New Deployments)

```
oneAgent:
  classicFullStack:
    # OneAgent DaemonSet provides code modules via host mounts
```

Pros	Cons
Simpler architecture	All pods share OneAgent
Single component	Requires hostPath mounts

applicationMonitoring

```
oneAgent:
  applicationMonitoring:
    # Only application-level monitoring, no infrastructure
    useCSIDriver: true
```

Pros	Cons
Minimal footprint	No infrastructure visibility
No node-level access needed	No container metrics

Pros	Cons
Good for managed K8s	

hostMonitoring

```
oneAgent:
  hostMonitoring: {}
```

Pros	Cons
Infrastructure only	No application tracing
Lightweight	No code-level insight

6. Configuration Options

Resource Limits

```
spec:
  oneAgent:
    cloudNativeFullStack:
      resources:
        requests:
          cpu: 100m
          memory: 256Mi
        limits:
          cpu: 300m
          memory: 512Mi
```

Node Selectors and Tolerations

```
spec:
  oneAgent:
    cloudNativeFullStack:
      nodeSelector:
        kubernetes.io/os: linux
      tolerations:
        - key: "dedicated"
          operator: "Equal"
          value: "monitoring"
          effect: "NoSchedule"
```

Namespace Selectors

Control which namespaces get injection:

```
spec:
  namespaceSelector:
    matchLabels:
      dynatrace-injection: enabled
```

Or exclude specific namespaces:

```
spec:
  namespaceSelector:
    matchExpressions:
      - key: dynatrace-injection
        operator: NotIn
        values:
          - disabled
```

Custom ActiveGate Configuration

```
spec:
  activeGate:
    capabilities:
      - kubernetes-monitoring
      - routing
    replicas: 2
    resources:
      requests:
        cpu: 500m
        memory: 512Mi
      limits:
        cpu: 1000m
        memory: 1Gi
    topologySpreadConstraints:
      - maxSkew: 1
        topologyKey: topology.kubernetes.io/zone
        whenUnsatisfiable: ScheduleAnyway
```

Feature Flags and Annotations

Control injection at the pod level:

Annotation	Effect
<code>oneagent.dynatrace.com/inject: "false"</code>	Disable injection for pod
<code>oneagent.dynatrace.com/inject: "true"</code>	Force enable injection
<code>oneagent.dynatrace.com/technologies: "java,nodejs"</code>	Limit technologies

DynaKube feature flags:

```
spec:
  oneAgent:
    cloudNativeFullStack:
      # Note: ONEAGENT_ENABLE_VOLUME_STORAGE defaults to true when CSI driver
      # is enabled – no need to set explicitly
```

Templates Configuration

Configure OTel Collector and Log Monitoring via `spec.templates` :

```
spec:
  templates:
    otelCollector:
      # ⚠️ Always pin to specific version - avoid 'latest'.
      # Current latest as of May 2026 is v0.48.0; verify against the live
      releases:
        # https://github.com/Dynatrace/dynatrace-otel-collector/releases
        imageRef:
          repository: public.ecr.aws/dynatrace/dynatrace-otel-collector
          tag: "0.48.0"
        resources:
          requests:
            cpu: 100m
            memory: 256Mi
          limits:
            cpu: 500m
            memory: 512Mi

    logMonitoring:
      resources:
        requests:
          cpu: 100m
          memory: 256Mi
        limits:
          cpu: 500m
          memory: 512Mi
      tolerations:
        - effect: NoSchedule
          key: node-role.kubernetes.io/control-plane
          operator: Exists
```

Note: Log monitoring is enabled with `spec.logMonitoring: {}` but resource configuration goes under `spec.templates.logMonitoring`.

OTel Collector and Log Monitoring Sizing Guide

Environment	CPU Limit	Memory Limit	Notes
Dev/Test	200m	256Mi	Low telemetry volume
Staging	500m	512Mi	Medium volume
Production	1000m	1Gi	High volume

7. Verification and Validation

Check Operator Status

```
# Operator pods
kubectl -n dynatrace get pods -l app.kubernetes.io/name=dynatrace-operator

# DynaKube status
kubectl -n dynatrace get dynakube

# Detailed status
kubectl -n dynatrace describe dynakube dynakube
```

Check OneAgent Deployment

```
# OneAgent DaemonSet
kubectl -n dynatrace get daemonset -l app.kubernetes.io/component=oneagent

# OneAgent pods on each node
kubectl -n dynatrace get pods -l app.kubernetes.io/component=oneagent -o wide
```

Check ActiveGate

```
# ActiveGate StatefulSet
kubectl -n dynatrace get statefulset -l app.kubernetes.io/component=activegate

# ActiveGate pods
kubectl -n dynatrace get pods -l app.kubernetes.io/component=activegate
```

DynaKube Status Phases

Phase	Meaning
Running	All components healthy
Deploying	Components being created
Error	Check events for details

```
// Verify Kubernetes cluster is reporting to Dynatrace
fetch dt.entity.kubernetes_cluster
| fields entity.name, tags
| sort entity.name asc

// Smartscape equivalent (dt.entity.* is deprecated but still functional):
//   smartscapeNodes "K8S_CLUSTER"
//   | fields name, tags
//   | sort name asc
// Caveat: Smartscape reflects CURRENT live topology and can report fewer
// entities than
// the classic entity store; for a pre-migration inventory keep the classic
// query above.
// Note: entity tags are not a flat "tags" field on Smartscape (resolve via
// getNodeField).
```

```
// Check OneAgent deployment health via events
fetch logs, from:-1h
| filter matchesPhrase(content, "dynatrace") and matchesPhrase(content,
"oneagent")
| fields timestamp, content
| sort timestamp desc
| limit 20
```

8. Upgrading the Operator

Helm Upgrade Process

```
# Update Helm repo
helm repo update

# Check current version
helm list -n dynatrace

# Upgrade to latest
helm upgrade dynatrace-operator dynatrace/dynatrace-operator \
  --namespace dynatrace \
  --reuse-values \
  --wait
```

Version Compatibility

Operator Version	Min K8s	Max K8s	Notes
1.7.x	1.23	1.29	Supported (verify against your environment)
1.8.x	1.24	1.30	Supported
1.9.x	1.25	1.31	Latest (v1.9.0, April 2026) — verify min/max ranges in the release notes for your specific tag

Operator support window: the Standard 9-month / Enterprise 12-month support policy means versions older than v1.7 are likely past EOL. Plan upgrades accordingly. Older versions may still function but will not receive security or compatibility fixes.

Rollback if Needed

```
# View history
helm history dynatrace-operator -n dynatrace

# Rollback to previous version
helm rollback dynatrace-operator 1 -n dynatrace
```

Next Steps

Now that the operator is deployed, proceed to:

Next Notebook	Topic
K8S-03: GitOps for DynaKube	Manage DynaKube with ArgoCD/Flux
K8S-04: Cluster Health Monitoring	Deep-dive into cluster metrics
K8S-05: Workload Monitoring	Application-level observability

Summary

In this notebook, you learned:

- Dynatrace Operator architecture and components
 - Prerequisites: API tokens and namespace setup
 - Helm chart installation with custom values
 - DynaKube CR structure and configuration
 - Deployment modes and when to use each
 - Configuration options for resources, selectors, and features
 - Verification commands and status phases
 - Upgrade and rollback procedures
-

References

- [Setup on Kubernetes — deployment \(DT docs\)](#) — application-observability, full-stack-observability, platform-observability deployment paths
 - [DynaKube parameters reference \(DT docs\)](#) — full CRD spec (renamed from `dynakube-crd`)
 - [DynaKube feature flags \(DT docs\)](#) — reference list of supported feature-flag annotations
 - [Helm chart values.yaml \(Dynatrace GitHub\)](#) — every Helm install option
 - [Dynatrace Operator releases \(Dynatrace GitHub\)](#) — current latest is v1.9.0 (April 2026); check before each install
-

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.